SkillTrend AI

A Technical Report on Skill-Demand Analysis and Forecasting from Job-Posting Data

Author: Savreet Kaur **Repository:** SkillTrend-AI **Report date:** 11 November 2026 **Version:** 1.0

Abstract

SkillTrend AI is a data-analysis and forecasting system that reads real job postings and answers three career decisions: which tools in a learner's stack are redundant, which in-demand skills they are missing, and where the demand for a given skill is heading over the next six months.

The system processes 58,276 cleaned job postings spanning November 2022 to April 2025, extracts 120 technical skills using a rule-based natural-language pipeline, aggregates demand as a monthly share of postings, forecasts each skill with Prophet against a naive baseline, and derives skill relationships from a positive pointwise mutual information co-occurrence graph. A second, independent live feed collects current postings from three free job-board APIs each month and profiles nine role families. Users may upload their own CSV of postings to move the forecast origin to the present day.

The report documents the full system: its architecture, every pipeline stage, the backend API, the frontend application, the testing strategy, the deployment configuration, and — in detail — two findings that changed the design. The first is a failed hypothesis: statistical co-occurrence provably cannot distinguish substitute tools from complementary ones. The second is a measurement artifact that initially appeared to be a market crash. Both are documented rather than hidden, because both shaped what the system does.

The entire project uses free and open-source software. It requires no paid services, no API keys, and no hosted database.

Index

Chapter 1 — Introduction . . . . . 5 1.1 The problem 1.2 What SkillTrend AI does 1.3 Who it is for 1.4 How this report is organised

Chapter 2 — Objectives and Scope . . . . . 7 2.1 Primary objectives 2.2 Explicit non-goals 2.3 Constraints 2.4 Success criteria

Chapter 3 — Background and Motivation . . . . . 9 3.1 Why "trending skills" lists are not useful 3.2 The three decisions that matter 3.3 Why job postings are a reasonable proxy 3.4 What postings cannot tell us

Chapter 4 — System Architecture . . . . . 11 4.1 The train-offline / serve-online split 4.2 Component overview 4.3 Data flow 4.4 Why Parquet 4.5 Directory layout

Chapter 5 — Data Sources . . . . . 14 5.1 The historical corpus 5.2 The live multi-role feed 5.3 User uploads 5.4 Why the three are never mixed 5.5 How the dataset was chosen

Chapter 6 — Stage 1: Loading and the Data Gate . . . . . 17 6.1 Purpose 6.2 What is inspected 6.3 Findings 6.4 An assumption that proved wrong

Chapter 7 — Stage 2: Cleaning . . . . . 19 7.1 Deduplication 7.2 Date parsing 7.3 Text normalisation 7.4 Results

Chapter 8 — Stage 3: Skill Extraction . . . . . 21 8.1 Why a dictionary and not a language model 8.2 Why PhraseMatcher and not regular expressions 8.3 The skill dictionary 8.4 The three disambiguation rules 8.5 Measured results 8.6 The test suite

Chapter 9 — Stage 4: Aggregation and the Month-Quality Gate . . . . . 25 9.1 Why percentages and not counts 9.2 The gate 9.3 The finding that justified it 9.4 Category aggregation and a double-counting bug

Chapter 10 — Stage 5: Trend Analysis . . . . . 28 10.1 Rolling averages 10.2 Period comparison 10.3 Direction classification 10.4 Results

Chapter 11 — Stage 6: Forecasting . . . . . 30 11.1 Model choice 11.2 Configuration and why seasonality is disabled 11.3 The backtest 11.4 Results, including the losses 11.5 Honest reporting of weak models

Chapter 12 — Stage 7: Co-occurrence and a Failed Hypothesis . . . . . 34 12.1 The goal 12.2 The hypothesis 12.3 The method 12.4 The measurement 12.5 Why it failed 12.6 What was kept

Chapter 13 — Stage 8: Families and Ranking . . . . . 38 13.1 Curated families 13.2 The ranking formula 13.3 A second correction: legacy bias 13.4 Disagreement warnings

Chapter 14 — The Live Multi-Role Feed . . . . . 41 14.1 Why a second source 14.2 Source selection 14.3 Role classification 14.4 Storage growth control 14.5 Why there are no role trends

Chapter 15 — The Backend API . . . . . 45 15.1 Design principles 15.2 Application structure 15.3 The data store 15.4 Endpoint reference 15.5 Error handling

Chapter 16 — The Decide Engine . . . . . 48 16.1 Four questions in order 16.2 Redundancy detection 16.3 Gap detection 16.4 Attribution and de-duplication 16.5 Worked example

Chapter 17 — Upload and On-Demand Forecasting . . . . . 52 17.1 Why uploads exist 17.2 Validation and limits 17.3 Column auto-detection 17.4 Append versus standalone 17.5 The refusal path 17.6 Prophet as an optional dependency

Chapter 18 — The Frontend . . . . . 56 18.1 Stack and rationale 18.2 Page structure 18.3 State management 18.4 Charting decisions 18.5 Pages removed in the redesign

Chapter 19 — Testing and Verification . . . . . 60 19.1 Unit tests 19.2 Validation suites inside the pipeline 19.3 API verification 19.4 Frontend verification

Chapter 20 — Deployment . . . . . 63 20.1 Deployment strategy 20.2 Backend on Render 20.3 Frontend on Vercel 20.4 Environment configuration 20.5 Scheduled data collection 20.6 Known deployment limits

Chapter 21 — Limitations 66

Chapter 22 — Results and Evaluation 69

Chapter 23 — Future Work 71

Chapter 24 — Conclusion 73

Appendix A — API Reference . . . . . 74 **Appendix B — Skill Dictionary** . . . . . 76 **Appendix C — Substitute Families** . . . . . 77 **Appendix D — Monthly Data Quality** . . . . . 78 **Appendix E — Glossary** 79

Chapter 1 — Introduction

1.1 The problem

Anyone learning technical skills faces the same three questions, and none of them are answered by the material that is easiest to find.

The first question is **what to learn next**. The usual answer is a listicle: "Top 10 skills for 2026". These are typically assembled from opinion, from a vendor's marketing interest, or from a single survey of self-selected respondents. They rarely state a sample size, almost never state a time window, and never show the underlying measurement.

The second question is **what not to learn**. This one is almost never addressed at all. A learner who already knows Tableau gains very little by learning Power BI; the two tools solve the same problem with different menus. Time spent on the second is time not spent on something genuinely new. But every "skills to learn" list treats skills as independent items to accumulate, so a diligent reader ends up collecting redundant tools.

The third question is **where demand is heading**. A skill that is common today but falling is a worse investment than a skill that is less common but rising. Answering this requires a time series, and a time series requires consistent measurement across months — which is exactly what casual data collection does not provide.

1.2 What SkillTrend AI does

SkillTrend AI answers all three questions from a measured corpus of real job postings, and shows the numbers behind every answer.

Given a stack of skills a user already has, the system returns:

- **Redundancy verdicts.** Which of the user's tools do the same job, which single member of that group is worth keeping, and what the evidence is.
- **Gap recommendations.** Which skills appear most often in postings that want the user's existing stack, expressed as a conditional probability.
- **Decline warnings.** Which held skills are measurably falling in demand.

Separately, for any individual skill, the system returns a six-month forecast with an uncertainty interval, accompanied by that model's own backtested accuracy — including cases where the model performs worse than a naive baseline.

A user may also upload their own CSV of job postings. The system cleans it, runs the same extractor over it, and refits a forecast so the projection starts from the user's most recent month rather than from the end of the historical corpus.

1.3 Who it is for

The primary user is someone planning what to learn: a student, a career changer, or a working engineer choosing between two directions. The system assumes they can read a percentage and a trend line, and does not assume any statistical training beyond that.

A secondary audience is anyone evaluating the project as engineering work. For that reader the system is designed to be auditable: every number traces to a script, every script states its assumptions in its header, and the limitations are a documented page in the application rather than a footnote.

1.4 How this report is organised

Chapters 1 to 3 set out the problem and the scope. Chapter 4 describes the architecture. Chapters 5 to 14 walk through the data pipeline in execution order, one chapter per stage, each explaining what the stage does, why it was built that way, and what it produced. Chapters 15 to 18 cover the serving layer: the API, the decision engine, the upload path, and the user interface. Chapters 19 and 20 cover testing and deployment. Chapters 21 to 24 assess results, state limitations honestly, and conclude. Five appendices hold reference material.

Two chapters are longer than their stage warrants, deliberately. Chapter 9 documents a measurement artifact that nearly produced a false headline finding, and Chapter 12 documents a hypothesis that failed. Both are included at

length because both changed the design of the system, and because a report that records only the parts that worked is not an accurate account of the work.

Chapter 2 — Objectives and Scope

2.1 Primary objectives

The project set out to achieve six things.

O1. Measure skill demand from primary data. Not from surveys or opinion, but from the text of job postings, with the extraction method stated and testable.

O2. Make the measurement comparable across time. Monthly posting volume in the source corpus varies by a factor of thirteen. Any metric that does not normalise for this will report collection artifacts as market movements.

O3. Forecast demand, and report the forecast's accuracy alongside it. A projection without an error estimate is a guess presented as a result. Every forecast in the system is backtested and carries its own metrics.

O4. Distinguish substitutes from complements. The system must be able to say "you do not need both of these" as well as "you are missing this".

O5. Allow analysis of the user's own data. The historical corpus ends in 2025. A user with current postings should be able to move the forecast origin to the present.

O6. Be deployable for free. The complete system must run on free hosting tiers with no paid dependency of any kind.

2.2 Explicit non-goals

Several plausible features were deliberately excluded.

No résumé parsing, no job matching, no application tracking. These belong to a different kind of product. The system's scope ends at the learning decision.

No salary analysis. The source corpus has salary data for a minority of postings, and that minority is not randomly distributed — remote and senior roles disclose more often. An analysis built on it would be systematically biased in a way the user could not see.

No company-level analysis. Posting volume by company measures hiring velocity and posting practice, not skill demand.

No role-level trends or forecasts. The live feed has no history: job boards list only current openings. Drawing a trend through two months of collection would be inventing a result. This is revisited in Chapter 14.

No large language model in the extraction path. Discussed in Chapter 8.

2.3 Constraints

Three constraints were fixed at the start and held throughout.

Free software only. No paid APIs, no paid hosting, no commercial datasets, no licensed tooling. An OpenAI key was available and was deliberately not used, because a dependency on it would make the project unrunnable for anyone else.

Deployable. The project had to end as something with a URL, not only as scripts on one machine. This constrained artifact sizes, dependency weight, and memory footprint throughout.

Auditable over clever. Where a simple explainable method and a complex opaque one were both available, the explainable one was chosen, provided it performed comparably. Chapter 8 is the clearest instance of this.

2.4 Success criteria

The project was judged against five measurable criteria.

#	CRITERION	TARGET	RESULT
C1	Extraction coverage	Beat the dataset's own skill labels	82.16% vs 78.60%
C2	Extraction precision	All disambiguation tests pass	8 of 8
C3	Forecast honesty	Every forecast reports a backtest	88 of 88
C4	Relationship validation	Complements and unrelated pairs correct	6/6 and 3/3
C5	Deployability	Serving artifacts under 1 MB	440 KB

Criterion C1 is worth noting: the source dataset ships with its own extracted skill labels, which gives an independent baseline to beat rather than a number to report in isolation. The system beats it while using a smaller vocabulary — 120 terms against 134.

Chapter 3 — Background and Motivation

3.1 Why "trending skills" lists are not useful

The standard artifact in this space is a ranked list of skills published annually. Three structural problems make these hard to act on.

They measure popularity, not marginal value. A list that says SQL is the most-requested skill is correct and useless: a reader who already knows SQL learns nothing, and a reader who does not is not told what to pair it with.

They treat skills as independent. Tableau and Power BI typically both appear in the top twenty. A reader following the list learns both, and gains far less from the second than from a skill in a different category.

They have no error bars. A ranking derived from one survey in one quarter is presented with the same confidence as one derived from a large corpus over years. The reader cannot distinguish them.

3.2 The three decisions that matter

The system is organised around decisions rather than around statistics. This reorientation came part-way through the project and resulted in removing two pages of descriptive dashboards that answered no question anyone had.

The three decisions are:

1. **"I have these skills — what is redundant?"** Answered by substitute families plus demand ranking.
2. **"I have these skills — what should I add?"** Answered by conditional co-occurrence probability.
3. **"Is this skill worth learning now?"** Answered by trend classification plus a backtested forecast.

Every page in the final application serves one of these three, or explains how they were computed.

3.3 Why job postings are a reasonable proxy

Job postings are not a perfect measure of skill demand, but they have three properties that make them a defensible one.

They are revealed preference, not stated preference. An employer writing a posting is committing budget. This is a stronger signal than a survey response.

They are dated. Each posting carries a collection date, which permits time series construction. Most alternative sources — curricula, course catalogues, documentation traffic — do not.

They are textual and specific. Postings name tools explicitly, which makes dictionary extraction viable with high precision.

3.4 What postings cannot tell us

Four limitations follow directly from the source and are carried through the entire report.

Mentions are not requirements. "Python a plus" and "expert-level Python required" are counted identically. No weighting scheme was attempted, because any such scheme would require labelled training data that does not exist for this corpus.

Postings are not hires. A posting may be filled, cancelled, or reposted. Volume measures hiring intent, not employment.

Posting text reflects templates. Large employers reuse boilerplate, so a single template can inflate a skill's apparent frequency across many postings. Deduplication (Chapter 7) removes exact repeats but not paraphrases.

The corpus has a role bias. Every posting in the historical corpus is a data-analyst advertisement. This is the single largest limitation of the project and is stated wherever a conclusion depends on it.

Chapter 4 — System Architecture

4.1 The train-offline / serve-online split

The system is divided into two halves that never run at the same time.

The **offline pipeline** is a sequence of ten Python scripts. It reads the raw corpus, performs all expensive computation, and writes a set of small Parquet and JSON artifacts. It is run manually or on a schedule, takes several minutes, and may consume substantial memory.

The **online API** reads only those artifacts. It performs no extraction, no model fitting, and no aggregation at request time, with a single deliberate exception: the user-upload path, which must compute on demand because the data did not exist until the request arrived.

This split is not stylistic. Fitting 88 Prophet models takes roughly three minutes; doing that inside an HTTP request is not viable under any timeout, and doing it per request would repeat identical work. The same division appears in production machine-learning systems for the same reason, and adopting it here was a deliberate decision to follow that pattern rather than to serve a model directly from a notebook.

A practical consequence is that the deployed API is small and fast. It loads roughly 440 KB of artifacts once at startup and then answers from memory.

4.2 Component overview

COMPONENT	TECHNOLOGY	RESPONSIBILITY
Pipeline	Python, pandas, spaCy, Prophet	Extraction, aggregation, modelling
Artifacts	Parquet, JSON	Serialised results
API	FastAPI, Pydantic, Uvicorn	Read artifacts, serve typed JSON
Decide engine	Python	Stack analysis over the artifacts
Upload service	pandas, spaCy	On-demand analysis of user CSVs
Frontend	React, Vite, Tailwind, Recharts	Presentation
Scheduler	GitHub Actions	Monthly live collection

4.3 Data flow

The complete flow, from raw file to rendered page, is as follows.

```
data/raw/gsearch_jobs.csv          276 MB, 61,953 postings
        |
   01_load.py          inspect schema, dates, nulls — the Data Gate
   02_clean.py         dedupe, parse dates, normalise text
        |
data/processed/clean_postings.parquet         58,276 postings
        |
   03_extract.py       spaCy PhraseMatcher over 120 skills
        |
data/processed/posting_skills.parquet         256,768 mentions
        |
   04_aggregate.py     skill x month as a share; quality gate
   05_trends.py        rolling means, period comparison
   06_forecast.py      Prophet + naive baseline backtest
   09_cooccurrence.py  PPMI graph and its validation
   10_relations.py     curated families, data-driven ranking
        |
   small Parquet + JSON artifacts  (~440 KB total)
        |
   FastAPI loads once at boot  ->  React app
```

The live track runs in parallel and independently:

```
   07_fetch_live.py    Arbeitnow, Jobicy, Remotive APIs
        |
data/live/live_postings.parquet
        |
   08_roles.py         title -> role family, skills per role
        |
data/processed/role_profiles.parquet
```

4.4 Why Parquet

Three properties made Parquet the right intermediate format.

Columnar compression. The cleaned corpus is 126 MB as Parquet. The same data as CSV is approximately 276 MB. The serving artifacts compress far more aggressively still, because they are numeric and highly repetitive — the entire set that the API needs is 440 KB.

Preserved types. Dates stay dates and booleans stay booleans across the boundary between scripts. With CSV every stage would re-parse, and a date parsed differently in two scripts is a class of bug that is hard to see.

Fast selective reads. The API reads whole files at boot, so this matters less than it would otherwise, but it makes pipeline development iterations faster.

The committed serving artifacts being small enough to live in Git is what makes free deployment possible at all: a deployed instance clones the repository and starts. There is no database to provision, no data migration to run, and no download step that could fail.

4.5 Directory layout

```
backend/app/
  main.py                  app, CORS, router registration only
  core/config.py           every path; CORS origins from environment
  schemas/models.py        Pydantic response contracts
  services/
    data_store.py          loads Parquet once at boot
    advisor.py             the Decide logic
    forecast_service.py    on-demand fitting; Prophet optional
    upload_service.py      validate, clean, extract, merge
    extraction.py          shares the pipeline's SkillExtractor
  api/routes/              summary, skills, trends, forecast,
                           roles, stack, upload
frontend/src/
  pages/                   Decide, Forecast, Roles, About
  components/              Layout, UploadPanel, ui
  hooks/useApi.js          loading / error / data for every call
  services/api.js          every backend call; VITE_API_URL aware
scripts/                   01–10 pipeline, extractor, role
                           classifier, two test suites
skills/
  skills.yaml              120 skills, aliases, ambiguity rules
  substitutes.yaml         curated families and why they are curated
data/
  raw/ processed/          gitignored: large and regenerable
  live/                    committed: cannot be regenerated
  uploads/                 gitignored, transient
.github/workflows/         monthly live fetch
render.yaml                backend deployment definition
```

One detail in this layout caused a real bug and is worth recording. The `.gitignore` originally excluded `data/processed/` and then attempted to re-include specific artifacts beneath it. Git cannot re-include a file whose parent directory is excluded, so every negation silently did nothing and the serving artifacts were never committed. The fix was to exclude `data/processed/*` with a trailing wildcard instead, and the reason is now written as a comment in the file so it is not reintroduced.

Chapter 5 — Data Sources

5.1 The historical corpus

The primary dataset is *Data Analyst Job Postings* by Luke Barousse, distributed on Kaggle. It was collected by scraping Google job-search results on a recurring schedule.

PROPERTY	VALUE
Raw postings	61,953
Raw file size	276 MB (CSV)
Usable after cleaning	58,276
Date span	November 2022 – April 2025
Analysis-grade span	November 2022 – July 2024
Analysis-grade months	21
Role coverage	Data analyst and adjacent titles
Geography	Predominantly United States
Licence	Open, free to download with a Kaggle account

The corpus ships with its own extracted skill labels. These were not used as input, but they were used as a **baseline to beat**, which turned a subjective quality question into a measurable one (Chapter 8).

5.2 The live multi-role feed

The historical corpus has two shortcomings: it ends in 2025, and it covers one role. A second, independent source addresses both.

Three job boards publish free, unauthenticated JSON APIs:

SOURCE	POSTINGS COLLECTED	CHARACTER
Arbeitnow	4,294	General board, Europe-weighted
Jobicy	200	Remote roles
Remotive	10	Remote technology roles
Total	**4,504**	

Of these, 1,354 classify into a technology role family; the remainder are non-technology postings from Arbeitnow's general listings and are excluded from every role profile.

The feed accumulates one month at a time via a scheduled GitHub Actions workflow. At the time of writing it holds two months of collection.

5.3 User uploads

The third source is whatever the user provides. The upload path accepts a CSV of up to 50 MB and 200,000 rows, auto-detects a date column and a description column, and runs the same cleaning and extraction code as the pipeline.

This is the only path in the system that computes at request time, and it exists because it is the only way to answer "forecast from *my* data, starting now" rather than "forecast from a corpus that ended in 2024".

5.4 Why the three are never mixed

The historical corpus is 58,276 United States data-analyst advertisements from 2022 to 2024. The live feed is a few thousand mostly-European, mostly-remote postings across nine role families from 2026. Averaging them would produce a number that describes neither population.

The system therefore keeps them in separate artifacts, serves them through separate endpoints, and labels every response with its source. The only place they are deliberately combined is the family ranking of Chapter 13, where two separate signals are blended with explicit weights and the disagreement between them is surfaced to the user rather than averaged away.

The upload path offers an *append* mode that splices user data onto the historical corpus. This is a deliberate exception, and it is handled carefully: the join month is recorded, returned in the API response, and drawn in the user interface, because a discontinuity at the seam is a change of source rather than a change in the market.

5.5 How the dataset was chosen

Several candidate datasets were compared before the historical corpus was selected. The decision came down to a trade-off between breadth and history.

CANDIDATE	ROLES	HISTORY	DECISION
Google-search analyst scrape	1 role	29 months	**Selected**
Multi-role posting dumps	Many roles	6–9 months	Rejected
Aggregated skill surveys	Many	Annual only	Rejected

Forecasting requires history. Six to nine monthly observations cannot support a trend estimate, let alone a validated one, so the broader datasets were unusable for the project's central objective. The narrow dataset with 29 months was chosen, and the resulting role bias was accepted as a known limitation and later mitigated — not removed — by adding the live multi-role feed.

This trade-off is the single most consequential decision in the project, and it is the reason Chapter 21 leads with role bias rather than with anything else.

Chapter 6 — Stage 1: Loading and the Data Gate

6.1 Purpose

`scripts/01_load.py` reads the raw CSV and writes nothing. Its entire output is a report.

This is unusual enough to justify. The temptation with a new dataset is to start transforming it immediately, and the cost of that is discovering a structural problem three stages later, after decisions have already been built on it. A stage that only inspects — a *data gate* — forces every assumption to be checked before any code depends on it.

6.2 What is inspected

The script reports six things:

1. **Schema.** Every column, its dtype, and a sample of values.
2. **Row count.** 61,953.
3. **Date coverage.** The minimum, maximum, and the per-month distribution.
4. **Null rates.** Per column, as a percentage.
5. **Description length.** The distribution of character counts, which later turned out to be the most important statistic in the dataset.
6. **Duplicate candidates.** Exact repeats by job identifier and by text.

6.3 Findings

Four findings from this stage changed later code.

Monthly volume is extremely uneven. Postings per month range from 3,646 to 272, a factor of thirteen. This immediately ruled out raw counts as the demand metric and is the direct reason for the percentage-based approach of Chapter 9.

Description length varies systematically over time. Median description length is roughly 3,100 characters in 2023 and falls to 1,557 by early 2025. At the loading stage this looked like a curiosity. It later turned out to be the cause of an apparent market-wide collapse in skill demand, and is the subject of Chapter 9.

The salary columns are sparse and non-randomly missing. This confirmed the decision to exclude salary from scope.

The dataset's own skill labels are present. This provided the baseline described in Chapter 8.

6.4 An assumption that proved wrong

Before writing the cleaning stage, the plan included BeautifulSoup to strip HTML tags from description text, on the assumption that scraped job descriptions would contain markup.

The loading stage measured it instead of assuming: the descriptions contain **zero HTML tags**. The scraper had already extracted plain text.

The dependency was removed. This is a small instance of a general principle the project tried to hold to — measure the data before writing code that depends on what it contains — and it is recorded here because the habit paid off much more significantly two stages later.

Chapter 7 — Stage 2: Cleaning

7.1 Deduplication

Scraped corpora contain repeats: the same posting collected on consecutive days, reposted by the employer, or syndicated across boards.

Deduplication proceeds in two passes. The first removes exact duplicates on the job identifier. The second removes rows whose normalised description text is byte-identical to an earlier row, which catches syndication where identifiers differ.

3,177 duplicate rows were removed.

No fuzzy or near-duplicate matching was attempted. Paraphrased repostings therefore survive, and this is stated as a limitation in Chapter 21. Fuzzy deduplication over 60,000 documents is expensive, and an aggressive threshold risks removing genuinely distinct postings from the same employer — a cost that is invisible in the output and therefore dangerous.

7.2 Date parsing

Each posting carries a collection timestamp. These are parsed to proper datetimes, and rows whose dates fail to parse or fall outside a sane window are dropped.

The date in this corpus is **scrape time, not posting time** — the moment the collector saw the advertisement, which is within a day or so of publication. At monthly granularity this difference is immaterial, but it is recorded because it is the kind of detail that silently invalidates a daily or weekly analysis.

A related defect appeared later in the live feed rather than here: the Arbeitnow API returns Unix epoch integers rather than ISO strings, and parsing them as strings produced a corpus in which every posting was dated 1 January 1970. This is covered in Chapter 14.

7.3 Text normalisation

Three transformations are applied to description text:

1. **Whitespace collapse.** Runs of spaces, tabs and newlines become a single space, so that token boundaries are consistent for the matcher.
2. **URL removal.** Links contribute no skill information and can introduce false matches through domain names containing tool names.
3. **Stub removal.** Descriptions below a minimum character threshold are dropped, because a 40-character description cannot support extraction and contributes a false zero to every skill.

500 stub descriptions were removed.

Deliberately *not* applied: lowercasing, stemming, and stop-word removal. The matcher is case-insensitive by configuration, so lowercasing the stored text would only destroy information useful for debugging. Stemming would break exact tool names. Stop-word removal would break multi-word skill names such as "Statistical Analysis".

7.4 Results

STAGE	ROWS
Raw	61,953
After identifier and text deduplication	58,776
After stub removal	**58,276**
Retention	94.1%

The output is `data/processed/clean_postings.parquet`, 126 MB. This file is gitignored: it is large and fully regenerable from the raw CSV.

Chapter 8 — Stage 3: Skill Extraction

This chapter describes the component the rest of the system rests on. Every trend, every forecast and every recommendation is downstream of whether this stage correctly identifies which skills a posting asks for.

8.1 Why a dictionary and not a language model

An obvious alternative was to send each posting to a large language model and ask it to list the skills. A paid API key was available. It was not used, and the reasoning is worth setting out because it was a deliberate engineering decision rather than a constraint.

Determinism. The same posting must produce the same skills on every run. A pipeline whose output changes between executions cannot support a trend analysis, because a change in the output cannot be attributed to a change in the market.

Auditability. When the system claims Python appears in 32.66% of postings, it must be possible to point at the exact rule that produced each match. A model's output cannot be inspected this way.

Cost and reproducibility. 58,276 documents through a commercial API costs real money on every re-run and makes the project unrunnable for anyone without a key. The dictionary approach costs nothing and runs anywhere.

Speed. The dictionary extractor processes the full corpus in **18.9 seconds**. An API-based approach would take hours and require retry handling, rate limiting, and partial-failure recovery.

Measured adequacy. The decisive argument is that the simple method is good enough: it beats the dataset's own labels, as Section 8.5 shows. A more complex method would need to justify its complexity with a measurable gain, and no such gain was demonstrated.

The honest counterargument is that a dictionary cannot find a skill it does not list. That limitation is real, is stated in the application's own Method page, and is the first item in the future-work chapter.

8.2 Why PhraseMatcher and not regular expressions

Having chosen dictionary matching, the implementation could have been regular expressions. It is not. The reason is a class of failure that is easy to miss.

Regular-expression word boundaries (`\b`) are defined in terms of word characters, and `+`, `#` and `/` are not word characters. Consequently:

- `\bC\+\+\b` does not match `C++` as intended, because there is no word boundary after the second `+`.
- `\bC#\b` has the same defect.
- `\bCI/CD\b` is split by the slash.

These are not edge cases: `C++`, `C#` and `CI/CD` are all real entries in the dictionary.

spaCy's `PhraseMatcher` avoids the problem entirely by tokenising the pattern and the text with the *same* tokeniser and matching token sequences. If spaCy tokenises `C++` as a single token in a pattern, it tokenises it the same way in a document.

The matcher runs on `spacy.blank("en")` — the tokeniser only, with no statistical model. This means no model download is required, which keeps the install light enough for a free deployment tier and removes a ~50 MB dependency from the backend.

8.3 The skill dictionary

`skills/skills.yaml` defines 120 skills across 9 categories.

CATEGORY	SKILLS
Programming Languages	18
Databases & Warehouses	17
Data Engineering & Big Data	16
AI & Machine Learning	15
BI & Visualization	14
DevOps & Version Control	11
Office & Collaboration	11
Generative AI	10
Cloud Platforms	8
Total	**120**

Each entry has a canonical name and a list of aliases. The canonical name is matched in addition to its aliases unless explicitly excluded:

```
Python: [python3, "python 3"]
SQL:    ["t-sql", tsql, "pl/sql", plsql, "ansi sql"]
C++:    [cpp, "c/c++"]
Go:     [golang, "go programming", "go language"]
```

The dictionary was built from evidence rather than from intuition. Every candidate term was checked against the actual corpus, and terms appearing in fewer than roughly 30 postings were removed as unusable — a skill with ten mentions across 58,000 documents produces a monthly series that is entirely noise and cannot be forecast.

8.4 The three disambiguation rules

Three rules exist in `scripts/skill_extractor.py`. Each was added to fix a specific, reproducible failure found by a test, not anticipated in advance.

Rule 1 — Longest match wins

The string `no-sql` tokenises to three tokens: `no`, `-`, `sql`. A naive matcher reports **both** NoSQL and SQL for that posting, inflating SQL.

The fix uses `spacy.util.filter_spans`, which given a set of overlapping spans keeps the longest non-overlapping subset. NoSQL wins; the embedded SQL match is discarded.

Rule 2 — Some canonical names never match alone

"Go" is a programming language and also one of the most common verbs in English. A posting reading "go to our careers page" would count as a Go posting.

The evidence that this matters is quantitative: the dataset's *own* labels put Go at 3.77% of postings — above Java. That figure is not the language. It is the verb.

The dictionary therefore carries a `canonical_not_matched` list. For entries on it, the canonical name is skipped and only the unambiguous aliases are matched:

```
phrases = list(aliases or [])
if canonical not in skip_canonical:
    phrases.insert(0, canonical)
```

Only `golang`, `go programming` and `go language` count toward Go.

Rule 3 — Context blocklists

"Excel" is both a spreadsheet and a verb. "Candidates must excel at communication" is not a Microsoft Excel posting; "advanced Excel skills" is.

The dictionary carries `blocked_next_tokens` per skill. If the token immediately following a match is on that skill's blocklist, the match is rejected:

```
Microsoft Excel: [at, in, as, beyond]
```

The ordering of this rule relative to Rule 1 is subtle and was a real bug. The blocklist must be applied **before** `filter_spans`, not after. If a rejected long span reaches the filter, it can suppress a legitimate shorter span that overlaps it, and the posting loses a skill it genuinely mentions. The code carries a comment saying exactly this, because the correct ordering is not obvious from reading the function:

```
def _extract_from_doc(self, doc) -> set[str]:
    spans: list[Span] = []
    for match_id, start, end in self.matcher(doc):
        skill = self.nlp.vocab.strings[match_id]
        # Rule 3: context blocklist. Applied BEFORE filter_spans so a
        # rejected span can never suppress a legitimate shorter one.
        blocked = self.blocked_next.get(skill)
        if blocked and end < len(doc) and doc[end].lower_ in blocked:
            continue
        spans.append(Span(doc, start, end, label=skill))
    # Rule 1: keep only the longest non-overlapping spans.
    return {span.label_ for span in filter_spans(spans)}
```

8.5 Measured results

METRIC	VALUE
Postings processed	58,276
Skill mentions extracted	256,768
Distinct skills found	120 of 120
Mean skills per posting	4.41
Hit rate (postings with ≥1 skill)	**82.16%**
Dataset's own labels, same measure	**78.60%**
Dictionary size	120 terms
Dataset's label vocabulary	134 terms
Extraction time	18.9 seconds

Two observations.

First, the extractor beats the dataset's shipped labels by 3.56 percentage points **while using a smaller vocabulary**. The comparison is meaningful precisely because the baseline is independent — it was produced by someone else with a different method.

Second, `never_matched` is empty: every one of the 120 dictionary entries matched at least one posting. This confirms the evidence-based dictionary construction worked. A dictionary with dead entries would indicate terms added from intuition rather than from the corpus.

8.6 The test suite

`scripts/test_extractor.py` contains eight cases and all eight pass. Three of them originally failed, and each failure produced one of the rules above.

#	INPUT	EXPECTATION
1	"experience with Python and SQL"	Python, SQL
2	"no-sql databases"	NoSQL only, **not** SQL
3	"please go to our careers page"	no Go
4	"golang microservices"	Go
5	"must excel at communication"	no Microsoft Excel
6	"advanced Excel skills"	Microsoft Excel
7	"CI/CD pipelines, C++ and C#"	CI/CD, C++, C#
8	"familiarity with LLMs"	LLM (plural alias)

Case 8 is worth noting: the plural "LLMs" was initially missed because only the singular was listed. The fix was an alias, not a stemmer — adding stemming to fix one plural would have introduced unpredictable behaviour across all 120 entries.

These tests are the project's main defence against regression in its most important component. Any future dictionary change runs against them.

Chapter 9 — Stage 4: Aggregation and the Month-Quality Gate

9.1 Why percentages and not counts

Monthly posting volume in the corpus ranges from 3,646 to 272. If demand were measured as the raw number of postings mentioning a skill, every skill would appear to collapse in late 2024 — not because employers stopped wanting them, but because the scraper collected fewer postings.

The metric is therefore:

```
demand(skill, month) = postings mentioning skill in month
                       ---------------------------------- x 100
                            total postings in month
```

This makes months comparable. A skill at 30% in a 3,000-posting month and 30% in a 300-posting month has the same measured demand, which is correct: both say that three postings in ten want it.

The output is `monthly_demand.parquet`, one row per skill per month, carrying the count, the total, and the percentage.

9.2 The gate

Normalising by volume fixes one artifact but not another. A second gate excludes months that cannot support analysis at all:

```
MIN_POSTINGS = 1_000
MIN_MEDIAN_DESC_CHARS = 2_000

totals["is_low_volume"] = totals["n_postings"] < MIN_POSTINGS
totals["is_truncated"]  = totals["median_desc_chars"] < MIN_MEDIAN_DESC_CHARS
totals["is_analysis_grade"] = ~(totals["is_low_volume"] | totals["is_truncated"])
```

Two independent failure modes are caught.

Low volume. Below about a thousand postings, a month's percentages are dominated by sampling noise, and a rare skill's series becomes a step function between zero and one posting's worth of share.

Truncation. If descriptions are systematically shorter, fewer skills can be found in them, so *every* skill's measured share falls at once. This does not look like noise. It looks like a coordinated market event, which is precisely what makes it dangerous.

Of 29 months in the corpus, **21 are analysis-grade** (November 2022 to July 2024). Eight are excluded. Every trend and every forecast in the system uses only the 21.

9.3 The finding that justified it

This gate exists because of a specific and nearly-published error, and it is the most instructive episode in the project.

Before the gate, the trend report classified **29 skills as declining and only 2 as rising**. Presented as a result, this is dramatic: a broad collapse in demand for technical skills across the board.

It was false. The cause was in the data-collection process. Median description length had fallen from roughly 3,000 characters to about 1,600 in the later months, because the scraper was capturing truncated text. Shorter descriptions contain fewer skill mentions, so every skill's share fell simultaneously.

The tell was the uniformity. A genuine market shift does not move all 120 skills in the same direction at the same time; shifts are differential, with some tools gaining as others lose. A signal that moves everything together is almost always a measurement artifact, and the correct response is to suspect the instrument rather than report the finding.

The table of per-month description lengths in Appendix D shows the pattern clearly. The excluded months are not deleted — they are still drawn on the comparison chart in the application, shaded, with a note explaining why they do not contribute to conclusions. Hiding them from the analysis is correct; hiding them from the user is not.

9.4 Category aggregation and a double-counting bug

The aggregation stage also produces category-level demand, so the application can say what share of postings want any BI tool rather than any particular one.

The first implementation summed the skill-level mentions within a category. This produced the impossible result that the Databases category appeared in **131.99%** of postings.

The error is straightforward in hindsight: a posting mentioning PostgreSQL, MySQL and Oracle contributes three mentions but is one posting. Summing mentions counts it three times.

The fix computes category demand from **distinct job identifiers**:

```
# A posting that names three databases is still ONE posting.
cat = (skills_long.merge(categories, on="skill")
       .groupby(["month", "category"])["job_id"].nunique())
```

written to `category_monthly.parquet`.

The bug is recorded here because a percentage above 100 is a fortunate kind of error — it is visibly impossible, so it cannot be shipped unnoticed. Had the categories been smaller, the same mistake would have produced inflated but plausible numbers, and nothing would have caught it. This argued for the sanity-check assertions that were subsequently added to the aggregation stage.

Chapter 10 — Stage 5: Trend Analysis

10.1 Rolling averages

Raw monthly percentages are noisy even in analysis-grade months. A three-month centred rolling mean is computed per skill to smooth short-term variation without introducing the lag of a trailing mean.

Smoothing is used for display and for direction classification. It is **not** used as the input to forecasting: Prophet performs its own decomposition, and feeding it pre-smoothed data would understate the uncertainty it reports.

10.2 Period comparison

Each skill's analysis-grade window is split in half. The mean share in the recent half is compared with the mean in the earlier half, giving an absolute change in percentage points and a relative change.

Absolute change is the primary measure. A skill moving from 2% to 4% has doubled in relative terms, which sounds dramatic, but has gained two percentage points — meaningfully less consequential than SQL moving from 52% to 50%. Both numbers are stored; the interface leads with the absolute one.

10.3 Direction classification

Each skill receives one of four labels:

LABEL	RULE
rising	Change exceeds the threshold upward and the series is not dominated by volatility
declining	Change exceeds the threshold downward, same condition
stable	Change within the threshold
insufficient data	Too few analysis-grade observations to classify

The `insufficient data` category matters. Without it, a skill with four observations receives the same confident label as one with twenty-one. Fifteen skills fall into this class and are excluded from trend-based recommendations rather than being given a weak label.

10.4 Results

Across 120 skills:

DIRECTION	COUNT
stable	85
rising	11
declining	9
insufficient data	15

This distribution is itself a validation of Chapter 9's gate. Before the gate, the same computation returned 29 declining and 2 rising. After it, the distribution is dominated by *stable* — which is what a 21-month window of a mature job market should produce. Tool adoption does not turn over in under two years.

The ten highest-demand skills in the corpus:

SKILL	CATEGORY	SHARE	DIRECTION
SQL	Programming Languages	52.42%	stable
Python	Programming Languages	32.66%	stable
Microsoft Excel	Office & Collaboration	32.59%	stable
Statistical Analysis	AI & Machine Learning	31.05%	stable
Tableau	BI & Visualization	28.79%	stable
Power BI	BI & Visualization	27.48%	stable
R	Programming Languages	19.12%	stable
Machine Learning	AI & Machine Learning	11.14%	stable
Data Warehouse	Data Engineering	10.44%	declining
Data Modeling	Data Engineering	10.38%	stable

These figures describe data-analyst postings specifically. SQL at 52% is a property of that role, not of software engineering in general, and the application states this wherever the number appears.

Chapter 11 — Stage 6: Forecasting

11.1 Model choice

Prophet was selected for three reasons: it handles short series without extensive tuning, it produces uncertainty intervals natively, and it is open-source and free.

The alternatives considered were ARIMA, which requires per-series order selection that is difficult to automate defensibly across 88 skills, and a simple linear trend, which produces no uncertainty interval. An LSTM or similar neural approach was rejected outright: 21 observations per series is far below what such a model needs, and fitting one would produce confident-looking output with nothing behind it.

Only skills averaging at least 0.5% of postings are modelled. Below that threshold a monthly series is mostly zeros and single-posting spikes. **88 skills** meet the threshold.

11.2 Configuration and why seasonality is disabled

Yearly seasonality is switched off.

The reason is arithmetic. Twenty-one monthly observations is 1.75 annual cycles. Fitting a yearly seasonal component to less than two cycles means fitting noise and presenting it as a pattern. Prophet will do this without complaint and will produce plausible-looking seasonal wiggles, which is precisely the danger: the output looks more informative, not less.

Weekly and daily seasonality are irrelevant at monthly granularity and are also disabled. What remains is a trend with changepoints and an uncertainty interval — which is all 21 points can honestly support.

The forecast horizon is six months. Beyond that the intervals widen to the point of uselessness, and quoting a 24-month projection from 21 observations would misrepresent what the data supports.

11.3 The backtest

Every skill is backtested before its forecast is reported.

The procedure is a holdout: remove the last three months, fit on the remaining eighteen, predict the held-out three, and compare the prediction against what actually happened. The same held-out months are predicted by a **naive baseline** that simply carries the last observed value forward.

Two metrics are recorded per skill: mean absolute error in percentage points, and mean absolute percentage error.

The naive baseline is the critical component. Without it, a reported MAE of 0.78 percentage points is uninterpretable — it could be excellent or embarrassing. Against a baseline it becomes meaningful: it tells the reader whether the model has learned anything that simply assuming no change does not already capture.

11.4 Results, including the losses

METRIC	VALUE
Skills modelled	88
Holdout	3 months
History per skill	21 months
Prophet mean MAE	**0.783 points**
Naive baseline mean MAE	**0.811 points**
Skills where Prophet wins	47 of 88
Win rate	**53.4%**

The honest reading is that Prophet beats a naive baseline, but barely. A 53% win rate is a coin flip with a slight lean. The aggregate improvement is 0.03 percentage points of error.

Per-skill results vary widely in both directions:

SKILL	PROPHET MAE	NAIVE MAE	VERDICT
Data Modeling	0.301	1.452	Prophet better by 79%
Scikit-learn	0.075	0.350	Prophet better by 79%
Kubernetes	0.315	0.915	Prophet better by 66%
Databricks	0.657	1.781	Prophet better by 63%
Python	1.812	1.837	Prophet better by 1%
SQL	3.531	2.514	**Naive better by 41%**
Machine Learning	1.287	0.837	**Naive better by 54%**
NLP	0.590	0.124	**Naive better by 375%**
SAP	1.881	0.409	**Naive better by 361%**

The large losses have a consistent cause: skills with low, flat series where the naive baseline is essentially perfect and Prophet's changepoint detection finds structure that is not there. NLP sits near 2% of postings with little movement; carrying the last value forward is very hard to beat, and attempting to model it adds error.

11.5 Honest reporting of weak models

A system could report only the aggregate — "Prophet beats the baseline" — which is true and misleading.

Instead, **every forecast response carries its own backtest metrics**, and the interface displays them beside the chart. A user looking at the SQL forecast sees that for SQL specifically the model loses to the naive baseline by 41%, with a caveat stating so in words.

This was a deliberate design choice with a real cost: it makes the product look less impressive. It is nonetheless the correct behaviour. A forecast interface that hides its own error rates is presenting guesses as knowledge, and a user acting on a bad SQL forecast is worse off than a user who was told not to trust it.

The interface reinforces this visually: observed history is drawn as a solid line and the forecast as a dashed line with a shaded 80% interval, because one is data and the other is a projection, and they should not look alike.

Chapter 12 — Stage 7: Co-occurrence and a Failed Hypothesis

This chapter documents a hypothesis that did not work. It is included in full because the failure determined the architecture of the Decide engine, and because a report that records only successes is not an accurate account of the work.

12.1 The goal

To advise on learning, the system needs to distinguish two relationships:

- **Substitutes** — tools that do the same job. Learning both largely wastes time. Tableau and Power BI; TensorFlow and PyTorch.
- **Complements** — tools that work together. Learning one increases the value of the other. Docker and Kubernetes; Python and Pandas.

Getting this backwards produces actively harmful advice. Telling a user to drop Kubernetes because they know Docker would be a serious error.

12.2 The hypothesis

The hypothesis was that the distinction is visible in co-occurrence statistics.

Specifically: substitutes should **repel**. An employer wants Tableau *or* Power BI, not both, so the two should appear together *less* often than chance would predict. Complements should attract, appearing together more often than chance.

This is intuitive, and it is wrong.

12.3 The method

A positive pointwise mutual information matrix was constructed over the 102 skills with at least 150 postings of support, across 47,860 postings containing at least one skill:

```
PMI(a,b)  = log( P(a,b) / (P(a) * P(b)) )
PPMI(a,b) = max(0, PMI(a,b))
```

PPMI measures how much more often two skills appear together than independence would predict. From this matrix four candidate signals were computed for each of 5,151 pairs: cosine similarity between PPMI vectors, lift, Jaccard index, and conditional asymmetry.

One refinement is worth noting. Cosine similarity between the PPMI rows of skills i and j is contaminated by the columns i and j themselves: two complements each have a high value in the other's column, which inflates their similarity. The implementation therefore uses a **masked cosine** that excludes both columns before comparing, so complements are not misread as substitutes through that artifact alone.

A labelled validation set of known pairs was assembled: eight known substitute pairs, six known complement pairs, and three known unrelated pairs.

12.4 The measurement

KNOWN PAIR TYPE	SIMILARITY	LIFT	JACCARD	ASYMMETRY
Substitutes	0.77	11.7	0.23	0.27
Complements	0.93	23.5	0.21	0.47

Substitutes do not repel. They score *lower* than complements on similarity and lift, but the distributions overlap so heavily that no threshold separates them.

Individual pairs make the overlap concrete:

PAIR	RELATIONSHIP	SIMILARITY	LIFT
TensorFlow / PyTorch	substitutes	0.96	57
Docker / Kubernetes	complements	0.97	69

These two pairs are statistically indistinguishable. Any threshold that classifies TensorFlow and PyTorch as substitutes also classifies Docker and Kubernetes as substitutes, and the system would advise dropping Kubernetes.

The validation result was recorded exactly as measured:

```
"validation": {
  "should be SUBSTITUTES": "0/8",
  "should be COMPLEMENTS": "6/6",
  "should be UNRELATED":   "3/3"
}
```

Zero of eight. The method failed completely at the task it was built for, while succeeding perfectly at the other two.

12.5 Why it failed

The reason is in how job advertisements are written. Employers do not write "must know Tableau". They write:

> *"experience with Tableau, Power BI or Looker"*

A posting listing alternatives contains all of them. From the corpus's point of view the substitutes co-occur constantly — more reliably, in fact, than many genuine complements, because they appear in the same enumerated list.

The deeper point is that the distinction between a substitute and a complement is **semantic**. It depends on what the tools *do*: whether one can replace the other functionally. Co-occurrence observes only which words appear near each other. The information required to make the distinction is not present in the signal, so no amount of refinement to the statistic would have recovered it. This is a limitation of the data, not of the implementation.

12.6 What was kept

Two of the three validation targets passed perfectly. The co-occurrence graph identifies complements at 6 of 6 and unrelated pairs at 3 of 3. That is exactly what gap detection needs: "what do employers who want your stack also want".

So the problem was split along what each method can actually do:

QUESTION	METHOD
"Are these alternatives?"	**Curated** — `skills/substitutes.yaml`
"Which one should I learn?"	**Data** — demand, trend, forecast, reach
"What am I missing?"	**Data** — co-occurrence, validated 6/6 and 3/3

Humans supply the meaning; data supplies the measurement. This is the same division already used by the skill dictionary, where a human declares that "golang" denotes Go and the corpus decides how often it appears.

Three decisions preserve the negative result rather than quietly discarding it:

1. The measured numbers are written into the **header of** `skills/substitutes.yaml`, so anyone reading the curated file immediately sees why it is curated.
2. The **validation re-runs on every pipeline execution**. If a future change makes statistical detection viable, the counts will change and the failure will stop being true.
3. The application's Method page **states the failure to users**, with the table, rather than presenting curated families as though they were derived.

Chapter 13 — Stage 8: Families and Ranking

13.1 Curated families

`skills/substitutes.yaml` declares ten families. Each carries a reason, written for a human reader.

FAMILY	MEMBERS
BI & dashboarding	Tableau, Power BI, Looker, Qlik, MicroStrategy, Domo, Cognos, Crystal Reports
Cloud platform	AWS, Azure, GCP
Cloud data warehouse	Snowflake, Redshift, BigQuery, Teradata, Azure Synapse
Relational database	PostgreSQL, MySQL, SQL Server, Oracle
Deep learning framework	TensorFlow, PyTorch
Statistics package	SAS, SPSS, MATLAB
Python plotting	Matplotlib, Seaborn, Plotly, ggplot2
NoSQL store	MongoDB, Cassandra, DynamoDB, Redis
Git hosting	GitHub, GitLab, Bitbucket
ETL tooling	Informatica, Talend, Fivetran, SSIS, Alteryx

Only unambiguous families are listed. Spark and Hadoop are **deliberately absent**: Spark largely replaced MapReduce, but the two still appear together in real stacks, and a wrong call there costs the user more than a missing one.

A second list, `not_substitutes`, names pairs that look like substitutes but are not, so the engine can never recommend dropping one of them:

PAIR	REASON
Docker / Kubernetes	Kubernetes orchestrates containers. Both are needed.
Python / Pandas	Pandas is a Python library, not an alternative.
Python / NumPy	Same — a library, not an alternative.
TensorFlow / Keras	Keras is TensorFlow's high-level API.
SQL / PostgreSQL	SQL is the language; PostgreSQL speaks it.
Machine Learning / Deep Learning	Deep learning is a subset.
ETL / Data Warehouse	ETL is the process that fills the warehouse.

13.2 The ranking formula

Within a family, which member is worth learning is answered entirely from data. Five weighted components produce a score:

```
W_DEMAND_HIST = 0.28   # share of the historical corpus
W_DEMAND_LIVE = 0.32   # share of current live postings
W_TREND       = 0.22   # direction and magnitude of change
W_REACH       = 0.13   # number of distinct role families wanting it
W_RISK        = 0.05   # penalty for high volatility
```

The live signal carries the largest single weight. This was not the original design, and Section 13.3 explains why it changed.

Reach deserves explanation: it counts how many of the nine role families in the live feed request the skill. A tool wanted by six role families is a safer investment than one wanted only by data analysts, because it remains useful if the learner's target role changes.

Risk is a small penalty for volatile series, reflecting that a skill whose share swings wildly is harder to plan around.

13.3 A second correction: legacy bias

The first implementation ranked family members using the historical corpus alone. It produced the following recommendations:

> *Learn **Oracle**. Learn **SAS**. Learn **Informatica**.*

Every one of these is *correct about that corpus*. The historical corpus is 2022–24 data-analyst postings from an enterprise-heavy United States market, and in that corpus Oracle genuinely outranks PostgreSQL.

Every one of them is also **bad advice for someone starting in 2026**. The system was accurately describing a population and silently presenting it as guidance about the present.

Blending the live multi-role feed into the score corrected it:

FAMILY	HISTORY ONLY	BLENDED	CHANGE
Relational database	Oracle	**PostgreSQL**	corrected
Deep learning framework	TensorFlow (tie)	**PyTorch**	corrected
ETL tooling	Informatica	**Fivetran**	corrected
BI & dashboarding	Power BI	Power BI	unchanged
Cloud platform	AWS	AWS	unchanged

Two observations. First, the corrections are concentrated exactly where technology has moved fastest since 2024 — databases, deep learning frameworks, and ingestion tooling. Second, the families where nothing changed are the ones where the two corpora agree, which is itself evidence that the blend is behaving sensibly rather than simply overwriting history with recency.

13.4 Disagreement warnings

Blending two signals can hide a disagreement inside an average. Where the two corpora rank a skill very differently, the system says so explicitly rather than smoothing it over:

> *⚠ Oracle ranks #1 in the 2022–24 corpus but #3 in current postings (7.2% then, 1.0% now).*

The threshold is a rank difference of two or more positions (`DISAGREEMENT_RANKS = 2`). This matters because the two corpora are genuinely different populations, and a large divergence is information — often the most useful information on the page, since it identifies exactly where a tool is being displaced.

Chapter 14 — The Live Multi-Role Feed

14.1 Why a second source

The historical corpus has two defects that no amount of processing can fix: it ends in April 2025, and every posting in it is a data-analyst advertisement.

A second source addresses both, but only if it is genuinely independent. The live feed is therefore built as a parallel track with its own scripts, its own storage, and its own endpoints. It never writes into the historical artifacts.

14.2 Source selection

The initial plan was to scrape Google job-search results, matching the historical corpus's own collection method. This was rejected for three reasons: it requires defeating bot detection, which is against the service's terms; commercial scraping platforms are paid, which violates a project constraint; and a browser-automation cron job is fragile and breaks within weeks of any layout change.

Three job boards publish free, unauthenticated JSON APIs instead. These are published by the boards themselves for exactly this purpose, need no key, and have stable contracts.

SOURCE	POSTINGS	NOTES
Arbeitnow	4,294	General board, Europe-weighted, deep pagination
Jobicy	200	Remote roles
Remotive	10	Remote technology roles

Two implementation defects were found and fixed during development.

Unix timestamps. Arbeitnow returns epoch integers rather than ISO date strings. Parsing them as strings produced a corpus in which every posting was dated 1 January 1970. A `unix_to_iso()` helper now handles both forms, and an `EARLIEST_SANE_DATE` guard set to 2015-01-01 rejects anything implausible — a check that would have caught the defect immediately had it existed first.

An ignored search parameter. The fetch originally passed `search=engineer` to narrow results to technology roles. Testing revealed that `search=engineer` and `search=nurse` return **byte-identical** responses: the parameter is accepted and ignored. Had this gone unnoticed, the feed would have been silently unfiltered while appearing filtered. The strategy changed to deep pagination (`ARBEITNOW_MAX_PAGES = 25`) followed by post-hoc classification.

14.3 Role classification

`scripts/roles.py` classifies a job title into one of nine role families using ordered regular-expression rules. The ordering is the entire mechanism.

"Senior Machine Learning Engineer" contains the word "Engineer". If Software Engineer were checked first, every machine-learning role would be misfiled. The rules therefore run from most specific to least specific and stop at the first match:

```
ROLE_RULES = [
    ("Machine Learning Engineer", [r"\bml engineer", r"\bmlops", ...]),
    ("Data Scientist",            [r"data scientist", ...]),
    ("Data Engineer",             [r"data engineer", r"analytics engineer", ...]),
    ("Data Analyst",              [r"data analyst", r"\bbi analyst", ...]),
    ("DevOps / SRE",              [r"\bdevops", r"site reliability", ...]),
    ("Security Engineer",         [r"security engineer", r"\bappsec", ...]),
    ("QA / Test Engineer",        [r"\bqa\b", r"\bsdet\b", ...]),
    ("Product Manager",           [r"product manager", r"\btpm\b", ...]),
    ("Software Engineer",         [...]),
]
```

Titles matching nothing become **"Other"** rather than being forced into a bucket. A wrong label is worse than an honest unknown: it would quietly corrupt the role comparison in a way nobody could see.

`scripts/test_roles.py` contains **26 tests**, all passing, and they specifically pin the ordering. If someone reorders the list, the tests fail.

Results across 4,504 live postings:

ROLE	POSTINGS	CONFIDENCE
Software Engineer	791	high
Product Manager	153	high
Machine Learning Engineer	90	low (<100)
DevOps / SRE	82	low
Data Engineer	60	low
Data Scientist	51	low
QA / Test Engineer	49	low
Security Engineer	41	low
Data Analyst	37	low
Other (non-technology)	*3,150*	excluded

Roles under 100 postings are marked low-confidence in the interface; under 30 they are dropped entirely. Approximately **70% of the feed is not technology work**, because Arbeitnow is a general job board. Those postings are classified Other and excluded from every profile.

14.4 Storage growth control

The live feed is committed to the repository, because unlike the historical corpus it **cannot be regenerated** — job boards serve only current openings, so a month not collected is lost permanently.

This creates a problem. Full posting text at the observed collection rate adds roughly 90 MB per year to a Git repository, which would make the project progressively slower to clone and eventually unusable on a free tier.

Two mechanisms control it:

1. **Description pruning.** Full description text is retained for three months (`DESCRIPTION_RETENTION_MONTHS = 3`) and then dropped. Old postings keep their metadata but lose their text.
2. **A persistent skill store.** `live_skills.parquet` holds the *extracted skills* per posting permanently. Because extraction happens before pruning, discarding the text loses nothing that the analysis needs.

This is the standard trade: keep the derived feature, discard the raw input. It makes indefinite accumulation viable within a repository.

14.5 Why there are no role trends

The Roles page shows current demand per role. It shows **no trends and no forecasts**, and this is a deliberate refusal rather than an unimplemented feature.

Job boards list only current openings. The feed has no history; it accumulates one month at a time and currently holds two months. Drawing a trend line through two points, or forecasting from them, would be inventing a result.

Once roughly twelve months have accumulated, the same trend machinery already built for the historical corpus can be applied to the role dimension without modification. Until then, the page states what it has and declines to extrapolate.

Chapter 15 — The Backend API

15.1 Design principles

Three principles shaped the backend.

Read-only at request time, with the single documented exception of uploads. Everything else is precomputed.

Typed contracts. Every response is a Pydantic model. The schema is generated automatically and is available at `/docs`, so the frontend is written against a specification rather than against observed behaviour.

No hardcoded paths. Every filesystem location lives in `backend/app/core/config.py`. Moving the data directory is a one-line change rather than a search through the codebase.

15.2 Application structure

`main.py` does exactly three things: creates the application, configures CORS, and registers routers. Keeping it thin is what prevents a FastAPI project from becoming one unreadable file.

Startup uses a lifespan context manager rather than the deprecated `@app.on_event("startup")` decorator:

```
@asynccontextmanager
async def lifespan(_: FastAPI):
    """Load the Parquet files once, at boot, rather than on first request."""
    store = get_store()
    if store.loaded:
        print(f"[data] loaded {len(store.skills)} skills, ...")
    else:
        print(f"[data] FAILED TO LOAD: {store.load_error}")
    yield
```

Note the failure behaviour: if artifacts fail to load, the application still starts. `/api/health` then reports `degraded` with the specific error. A server that refuses to start gives an operator nothing to inspect; one that starts and reports its own fault is diagnosable.

15.3 The data store

`services/data_store.py` loads every Parquet artifact once at boot into a singleton, which all routes share.

The artifacts the API needs total approximately **440 KB**. The heavy intermediate files — `clean_postings.parquet` at 126 MB and `posting_skills.parquet` at 46 MB — are pipeline-only and never deployed.

This is what makes the service viable on a free tier. Memory usage is a few megabytes, startup takes well under a second, and there is no database connection to manage.

15.4 Endpoint reference

Thirteen endpoints, all verified returning HTTP 200.

METHOD	PATH	RETURNS
GET	`/api/health`	Liveness, data status, active forecasting model
GET	`/api/summary`	Headline statistics and the data-quality report
GET	`/api/skills`	Searchable, sortable skill list
GET	`/api/trends`	Monthly series for up to 8 skills
GET	`/api/forecast/{skill}`	History, forecast, and backtest metrics
GET	`/api/families`	Substitute families with recommendations
GET	`/api/related/{skill}`	Substitutes and complements, scored
POST	`/api/stack/analyse`	A stack in, verdicts with evidence out
POST	`/api/upload`	A CSV in, an analysed dataset out
GET	`/api/upload/limits`	Size and row limits, for the UI
GET	`/api/roles`	Role profiles from the live feed
GET	`/api/roles/matrix`	Skill-by-role comparison matrix
GET	`/api/live/status`	Live-feed freshness and source counts

One routing defect is worth recording. The related-skills endpoint was originally `/api/skills/{skill:path}/related`. FastAPI's `path` converter is greedy, so it consumed the trailing `/related` as part of the skill name and the route never matched. Moving it to `/api/related/{skill:path}` fixed it. The `path` converter is required because skill names legitimately contain slashes — `CI/CD` must survive as a parameter.

15.5 Error handling

Three classes of error are distinguished.

Bad user input returns 400 with an actionable message. The upload path raises a dedicated `UploadError` whose message always states what the user should do differently, not merely that something failed.

Missing resources return 404 — a skill not in the dictionary, a role below the minimum threshold.

Unavailable optional data does not error. The live feed is optional; if its artifacts are absent the API starts normally and the roles endpoints report unavailability. An optional feature must not be able to take down the service.

One subtle bug in this area is worth recording. Pandas represents missing numeric values as `NaN`, and `bool(float("nan"))` is `True` in Python. A truthiness check intended to skip missing values therefore passed them straight through to Pydantic, which rejected them. The fix is an explicit `pd.isna()` check. Truthiness testing on values originating from pandas is unsafe wherever `NaN` is possible.

Chapter 16 — The Decide Engine

16.1 Four questions in order

`backend/app/services/advisor.py` answers four questions, in sequence:

1. **REDUNDANT** — Do you hold several tools that do the same job? *Source: curated families.*
2. **KEEP WHICH** — Within each, which is worth the time? *Source: measured demand, trend, reach.*
3. **MISSING** — What do employers wanting your stack also want? *Source: co-occurrence, P(skill | your stack).*
4. **DEAD WEIGHT** — Is anything you hold on the way out? *Source: trend direction.*

Every verdict carries the evidence it came from. Nothing in the engine says "consider learning X" without a number attached.

16.2 Redundancy detection

The engine maps each skill in the user's stack to its family, groups them, and raises a verdict for any family where the user holds more than one member.

The recommended member is the family's data-ranked winner (Chapter 13), unless the user already holds it — in which case the verdict simply advises dropping the others.

The `not_substitutes` list is consulted before any verdict is emitted, which guarantees the engine cannot recommend dropping Kubernetes because the user knows Docker.

16.3 Gap detection

Gaps come from conditional probability over the co-occurrence graph: of all postings that want *any* skill in the user's stack, what share also want some skill the user lacks?

Three thresholds control the output:

```
GAP_MIN_CONDITIONAL = 0.18   # at least 18% of relevant postings
GAP_MAX_RESULTS     = 8      # never overwhelm the user
GAP_MIN_SUPPORT_PCT = 0.8    # skill must be non-niche overall
```

`GAP_MIN_SUPPORT_PCT` prevents a specific failure: a rare skill can show a high conditional probability from a tiny absolute base. A skill appearing in 0.2% of all postings but 30% of postings mentioning some narrow tool is a statistical artifact, not a recommendation.

Gaps exclude **substitutes** of held skills — there is no point suggesting Power BI to someone who already knows Tableau — but deliberately do **not** exclude non-complements. The engine knows which pairs are substitutes; it does not claim to know every pair that is unrelated, and filtering on a judgement it cannot support would silently drop valid recommendations.

16.4 Attribution and de-duplication

Two refinements were added after the first implementation produced confusing output.

Attribution. A gap is attributed to the single stack skill with the highest conditional probability, so the user sees "R, in 51% of postings that want Python" rather than an unattributed percentage. Knowing *which* of their skills drives a recommendation is what makes it actionable.

Family collapse. Without this, a user holding Python would be told they are missing Tableau, Power BI, Looker and Qlik — four members of one family, presented as four separate gaps. The engine now collapses family members into a single gap represented by the family's recommended member.

16.5 Worked example

For the stack `Tableau · Power BI · Python · SQL`:

```
REDUNDANT   2 tools in your stack do the same job: Tableau, Power BI
            Keep Power BI — 27.5% of historical postings, 1.7% of
            current ones, rising (+2.02 points). Dropping Tableau
            frees up time without costing you coverage.

MISSING     5 skills commonly paired with your stack
            Statistical Analysis — in 52% of postings that want Python
            R                    — in 51%
            Microsoft Excel      — in 46% of postings that want Power BI

DECLINING   nothing in your stack is on the way out
```

Every line carries its number, and every number traces to a specific artifact produced by a specific pipeline stage.

Chapter 17 — Upload and On-Demand Forecasting

17.1 Why uploads exist

The historical corpus ends in April 2025, and only its first 21 months are analysis-grade. A forecast built on it projects forward from July 2024, which is of limited use to someone planning in 2026.

Uploads solve this directly: a user with current postings can move the forecast origin to their own latest month.

17.2 Validation and limits

Limits are enforced **before** anything is parsed, because the purpose of a limit is to avoid allocating memory for input that will be rejected:

```
MAX_BYTES  = 50 * 1024 * 1024   # 50 MB
MAX_ROWS   = 200_000
MIN_DESCRIPTION_CHARS = 100
MIN_MONTHS_TO_FORECAST = 12
```

These are sized for free hosting tiers rather than for what pandas could theoretically handle.

17.3 Column auto-detection

Requiring a user to name their date and description columns adds friction and invites error. The service infers both from column names:

```
DATE_HINTS = ("date", "posted", "created", "listed", "time", "published")
TEXT_HINTS = ("description", "job_description", "text", "summary",
              "details", "requirements", "jobdescription")
```

Detection is exact-match first, then substring. If no column matches, the error message lists the columns that *were* found, so the user can see what the service saw.

The uploaded data then passes through the **same cleaning and extraction code as the pipeline**. `services/extraction.py` adds `scripts/` to the Python path and imports the pipeline's `SkillExtractor` behind an `lru_cache`, rather than duplicating the logic. This matters: a second copy of the extractor would drift from the first, and uploaded data would be measured by different rules than the corpus it is compared against.

17.4 Append versus standalone

Standalone analyses the upload in isolation. Appropriate when the user's data is its own population.

Append splices the upload onto the end of the historical corpus so the forecast origin moves forward. This is the "predict from today" case, and it is the reason the feature exists.

Append is honest about what it is doing. The two datasets are different populations, so the **join month is recorded**, returned in the API response, and drawn in the interface. A jump at the seam is a change of source, not a change in the market, and the user must be able to see where it is.

17.5 The refusal path

If the data — whether the upload alone or the spliced series — contains fewer than twelve months of history, **no forecast is produced**.

The response explains why instead. This is a deliberate refusal: a trend fitted to five points will produce a confident-looking line, and that line is noise with a slope. Refusing is less impressive than producing output and is the correct behaviour.

An end-to-end test of the path: a 1,629-row CSV produced 1,510 usable rows across 14 months with a 100% extraction hit rate, columns detected automatically, and a forecast of 14 history months plus 6 projected months at MAE 2.92 against a naive baseline of 7.43.

17.6 Prophet as an optional dependency

Prophet with its compiled Stan binary is approximately 400 MB — more than free hosting tiers comfortably accommodate. It is therefore **excluded from** `backend/requirements.txt`, which contains runtime dependencies only.

The forecast service detects its availability at runtime:

```
def _prophet_available() -> bool:
    try:
        import prophet  # noqa: F401
        return True
    except Exception:
        return False
```

When Prophet is absent, on-demand forecasts fall back to an ordinary least-squares trend with an interval derived from residual spread (`INTERVAL_Z = 1.2816`, the 80% normal quantile). **The response states which model produced it**, and `/api/health` reports the active model.

The substitution is defensible because it was measured, not assumed. On Python, the fallback scores MAE **1.8154** against Prophet's **1.8124** — a difference of three thousandths of a percentage point. This is consistent with the finding of Chapter 11 that Prophet beats a naive baseline on only 53% of skills: when a sophisticated model barely beats carrying the last value forward, a linear trend is not a meaningful downgrade.

Batch forecasts for the historical corpus are precomputed and committed, so the deployed server never needs Prophet for its main path at all.

Chapter 18 — The Frontend

18.1 Stack and rationale

TECHNOLOGY	VERSION	REASON
React	19	Component model, large ecosystem
Vite	8	Fast builds, native ES modules
Tailwind CSS	4	Styling without a separate stylesheet to drift
Recharts	3	Declarative charts over React components
React Router	7	Client-side routing with URL state

Tailwind v4 is integrated through `@tailwindcss/vite` rather than the PostCSS plugin, which is the current recommended path and removes a configuration file.

18.2 Page structure

Four pages, each answering one question:

PAGE	QUESTION
Decide	What should I learn, and what is redundant?
Forecast	Where is this skill heading? Plus: upload your own data.
Roles	What does my target role actually want?
Method	How does this work, and where does it fail?

The Method page is not an afterthought. It contains the failed hypothesis of Chapter 12 with its table, the data-quality gate of Chapter 9 with the 29-declining incident, and the full limitations list. Putting this in the product rather than only in documentation is a deliberate statement about what the numbers are worth.

18.3 State management

No state-management library is used. React's built-in state plus a single custom hook covers every case.

`hooks/useApi.js` handles the three states every API call needs — loading, error, and data — in one place:

```
export function useApi(fetcher, deps = []) {
  const [data, setData] = useState(null);
  const [error, setError] = useState(null);
  const [loading, setLoading] = useState(true);
  ...
}
```

Two details matter. First, a cancellation flag guards against a slow response landing after the user has navigated away, which would otherwise overwrite new data with stale data. Second, centralising the error state means no page can forget it — a forgotten error state is how a blank white screen with no explanation happens.

URL parameters are the source of truth for selections. The Forecast page derives the selected skill from `useSearchParams` rather than mirroring it into component state. This was a correction: mirroring required an effect that set state on every parameter change, which cascades renders and triggered ESLint's `set-state-in-effect` rule. Deriving the value removed the effect entirely — and made forecast URLs shareable, which mirroring would not have.

Where an ESLint rule genuinely fires a false positive — the standard data-fetching pattern inside `useApi` — the rule is disabled with a written justification in a comment explaining why it does not apply, rather than silently.

18.4 Charting decisions

Three decisions govern every chart.

Animation is disabled. `isAnimationActive={false}` on every series. The draw-in animation left charts half-rendered on first paint, which is worse than no animation, and makes screenshots unreliable.

History and forecast look different. Observed data is a solid line; the forecast is dashed with a shaded 80% interval. One is measurement and the other is projection, and they must not be confusable.

Excluded months remain visible. Months failing the quality gate are drawn shaded rather than removed, with an explanation. They are excluded from conclusions, not from view.

18.5 Pages removed in the redesign

The application originally had six pages. Three were removed when the project was reoriented around decisions.

REMOVED	WHY
Dashboard	Descriptive statistics answering no decision
Skill Explorer	Browsing without a question to answer
Compare	Merged into Decide, which does it with a recommendation

Removing working code is harder than writing more. The test applied was whether each page answered one of the three decisions in Section 3.2. A page showing accurate numbers that nobody can act on is not a neutral addition — it costs attention and dilutes what the product is for.

Old routes redirect to `/` rather than 404, so any shared link still works.

Chapter 19 — Testing and Verification

19.1 Unit tests

SUITE	CASES	STATUS	COVERS
`test_extractor.py`	8	pass	Skill extraction and all three disambiguation rules
`test_roles.py`	26	pass	Title classification and rule ordering

Both suites run without a test framework dependency — plain Python with assertions, executable directly. This keeps the pipeline's dependency list short and means the tests run anywhere Python does.

Coverage is deliberately concentrated on the two components where an error is both likely and invisible downstream. A bug in extraction corrupts every number in the system silently; a bug in a route handler produces a visible error.

19.2 Validation suites inside the pipeline

Two validations run as part of normal pipeline execution, not as separate tests.

Relationship validation (Chapter 12) checks the co-occurrence graph against labelled pairs on every run and writes the result to `skill_relations.json`. The current result — 0/8 substitutes, 6/6 complements, 3/3 unrelated — is the documented negative result, and it stays visible because it is regenerated every time.

Data-quality gating (Chapter 9) recomputes which months are analysis-grade on every aggregation run. If a future data refresh changes the picture, the gate adapts rather than relying on a hardcoded month list.

19.3 API verification

All thirteen endpoints were verified returning HTTP 200 with valid payloads, including the awkward cases:

- `/api/related/CI%2FCD` — a URL-encoded slash in a skill name
- `/api/forecast/Python?dataset=<upload_id>` — on-demand fitting path
- `POST /api/stack/analyse` — the full Decide flow
- `POST /api/upload` — end-to-end, 1,629 rows through to a fitted forecast

The Prophet-optional path was verified by removing Prophet and confirming both that the fallback activates and that it reports itself as the active model.

19.4 Frontend verification

Verified in a browser rather than assumed:

CHECK	RESULT
ESLint	Exit 0, no warnings
Production build	Succeeds — 709 KB JS, 210 KB gzipped
All four pages render	No console errors
Old routes redirect	`/skills` and `/compare` → `/`
Upload flow end to end	14 months in, 6 months forecast out

Chapter 20 — Deployment

20.1 Deployment strategy

The constraint was free hosting with no database, which shaped three decisions.

Serving artifacts are committed. All 440 KB of them. A deployed instance clones the repository and starts: no pipeline run, no raw download, no migration, no data-provisioning step that could fail.

The backend dependency list is minimal. `backend/requirements.txt` excludes Prophet, Kaggle and Matplotlib, with a comment in the file explaining each exclusion.

Configuration comes from the environment. Neither half of the application requires a code change to deploy.

20.2 Backend on Render

`render.yaml` defines the service for Render's free tier, with the health check wired to `/api/health`. Any host works:

```
pip install -r backend/requirements.txt
cd backend && uvicorn app.main:app --host 0.0.0.0 --port $PORT
```

20.3 Frontend on Vercel

`vercel.json` and `public/_redirects` configure SPA rewrites so that React Router paths do not 404 on direct navigation — a static host asked for `/forecast` must serve `index.html` rather than looking for a file.

```
cd frontend && npm run build   # -> dist/
```

20.4 Environment configuration

VARIABLE	SIDE	PURPOSE
`SKILLTREND_CORS_ORIGINS`	Backend	Comma-separated frontend origins
`VITE_API_URL`	Frontend	Deployed API base URL

Both have working local defaults, so development requires no configuration:

```
CORS_ORIGINS = _DEFAULT_ORIGINS + [
    origin.strip()
    for origin in os.environ.get("SKILLTREND_CORS_ORIGINS", "").split(",")
    if origin.strip()
]
```

```
const API_BASE = `${import.meta.env.VITE_API_URL ?? "http://127.0.0.1:8000"}/api`;
```

20.5 Scheduled data collection

`.github/workflows/fetch-jobs.yml` runs the live fetch monthly on GitHub Actions and commits the result. This is free for public repositories and requires no server.

It is the mechanism by which the live feed accumulates the history it needs before role trends become possible.

20.6 Known deployment limits

Uploads are ephemeral. They are written to `data/uploads/`, which does not survive a restart on free tiers. Acceptable because uploads are transient by design, and stated rather than discovered.

Free tiers sleep. The first request after idle is slow. This is a property of the tier, not of the application.

Prophet is absent in production. On-demand forecasts use the OLS fallback and say so. Measured as near-identical in Chapter 17.

Chapter 21 — Limitations

Stated plainly, in rough order of how much they constrain conclusions. All of these also appear in the application's own Method page.

1. One role in the historical data. Every posting is a data-analyst advertisement. Gap advice is strongest for data tooling and weakest for web and DevOps work. This is a direct consequence of the dataset trade-off in Section 5.5 — 29 months of history was chosen over breadth, because forecasting requires history. It is the single largest limitation in the project.

2. One source for the historical data. Scraped from Google job-search results. Whatever that collection missed, this analysis misses.

3. Mentions are not requirements. "Python a plus" and "expert Python required" count identically. No weighting was attempted because no labelled data exists to validate one.

4. Forecasts are weak. 21 monthly observations is very little history. Prophet beats a naive baseline on 53% of skills. The honest reading of any forecast here is a direction, not a number.

5. Substitute families are human-curated, because the data provably could not derive them (Chapter 12). The curation is conservative — ambiguous families such as Spark/Hadoop are omitted — but it is judgement, not measurement.

6. A dictionary cannot find what it does not list. A skill absent from the 120 terms reads as 0%, which is indistinguishable from genuinely unwanted.

7. Scrape time, not posting time. Dates are when the collector saw the advertisement, within about a day of publication. Fine monthly, unsafe daily.

8. Role profiles are small. Software Engineer has 791 postings behind it; Data Analyst has 37. Roles under 100 are flagged low-confidence and under 30 are dropped, but even the flagged ones are thin.

9. Approximately 70% of the live feed is not technology work. Arbeitnow is a general job board. Non-technology postings are classified Other and excluded, but this means the effective technology sample is 1,354, not 4,504.

10. Append mode splices two populations. It is the only way to move the forecast origin to the present, and it is not the same as one continuous corpus. The join month is recorded and displayed for this reason.

11. Deduplication is exact, not fuzzy. Paraphrased repostings survive and can inflate a skill's apparent frequency.

12. Geographic skew. The historical corpus is predominantly United States; the live feed is Europe-weighted and remote-weighted. Neither is global.

Chapter 22 — Results and Evaluation

22.1 Against the stated objectives

#	OBJECTIVE	OUTCOME
O1	Measure demand from primary data	58,276 postings, 256,768 mentions, method tested
O2	Comparable across time	Percentage metric plus a quality gate; 21 of 29 months pass
O3	Forecast with reported accuracy	All 88 forecasts carry their own backtest
O4	Distinguish substitutes from complements	Partially: complements measured, substitutes curated
O5	Analyse the user's own data	Upload path, verified end to end
O6	Deployable for free	440 KB artifacts, no paid dependency

O4 is marked partial, which is accurate. The original intention was to derive both relationships from data, and that failed at 0 of 8 for substitutes. The delivered system achieves the user-facing goal through a different mechanism, and documents why.

22.2 Quantitative summary

MEASURE	VALUE
Raw postings	61,953
Cleaned postings	58,276 (94.1% retained)
Skills in dictionary	120 across 9 categories
Skill mentions extracted	256,768
Mean skills per posting	4.41
Extraction hit rate	82.16% (baseline 78.60%)
Extraction time	18.9 seconds
Analysis-grade months	21 of 29
Skills forecast	88
Prophet MAE / naive MAE	0.783 / 0.811
Prophet win rate	53.4%
Co-occurrence pairs	5,151 over 102 skills
Curated families	10
Live postings	4,504 (1,354 technology)
Role families profiled	9
API endpoints	13, all verified 200
Unit tests	34, all passing
Serving artifact size	~440 KB
Frontend bundle	709 KB, 210 KB gzipped

22.3 What the project demonstrates

Beyond the deliverable, three things were demonstrated in the work.

That measurement artifacts can masquerade as findings. The 29-declining incident of Chapter 9 would have been a plausible, dramatic, entirely false headline. Catching it required noticing that the uniformity of the signal was itself suspicious.

That a negative result can be load-bearing. The failure of Chapter 12 was not discarded; it determined the split between curated and derived knowledge that the Decide engine rests on, and it is preserved in the code, the validation suite, and the user interface.

That simple methods should be measured before being replaced. The dictionary extractor beats the dataset's own labels with a smaller vocabulary in 19 seconds and costs nothing. The case for replacing it with a language model was never established, so it was not replaced.

Chapter 23 — Future Work

Four extensions, ordered by value relative to effort.

1. A learned extractor, evaluated against the rule-based baseline. Use the dictionary as a weak-supervision labeller to generate training data, fine-tune a transformer for named-entity recognition over job descriptions, and compare F1 against the current extractor. This directly addresses Limitation 6 — a learned model can find skills absent from the dictionary. The rule-based version remains the baseline to beat, exactly as the dataset's labels served as the baseline for it.

2. Broaden the live feed. Adding Adzuna, which offers a free developer tier, would widen coverage beyond remote and European roles and improve the blended ranking of Chapter 13.

3. Role-dimension trends. Once roughly twelve months of live data have accumulated, the existing trend and forecast machinery applies to roles without modification. The constraint is calendar time, not engineering.

4. Weighted mentions. Distinguishing "required" from "nice to have" would address Limitation 3, but requires a labelled corpus to validate against. Without validation it would be a heuristic presented as a measurement, which is the failure mode this project has tried throughout to avoid.

Chapter 24 — Conclusion

SkillTrend AI answers three learning decisions from 58,276 real job postings, and shows the evidence for every answer.

The engineering is ordinary in its parts: a rule-based extractor, a percentage-normalised time series, Prophet with a backtest, a co-occurrence graph, a typed API, a small React application. What the project attempted to do well is the part that is usually left out — establishing whether each of those parts actually works, and reporting the answer when it does not.

Three results illustrate that.

The extractor was measured against the dataset's own labels and beat them, 82.16% against 78.60%, with a smaller vocabulary. That is a modest result, stated precisely, against an independent baseline.

The forecasts were backtested against a naive baseline and win 53% of the time. That is a weak result, and it is displayed next to every forecast in the product rather than summarised away.

The substitute-detection hypothesis failed completely, 0 of 8. That failure is recorded in the code, re-validated on every pipeline run, and shown to users on the Method page — and it is the reason the Decide engine splits curated meaning from measured ranking, which is the system's central design decision.

The project also caught two errors that would have shipped silently. A truncation artifact in the source data appeared as a market-wide collapse until the uniformity of the signal gave it away. A ranking built on historical data alone recommended Oracle, SAS and Informatica — accurate about a 2022–24 corpus, and wrong as advice for 2026.

The whole system runs on free and open-source software, requires no API keys, and deploys from a Git clone.

Appendix A — API Reference

METHOD	PATH	PARAMETERS	RETURNS
GET	`/api/health`	—	status, data_loaded, active forecast model
GET	`/api/summary`	—	headline stats, data-quality report, model quality
GET	`/api/skills`	`limit`, `sort`, `q`	skill list with share and direction
GET	`/api/trends`	`skills` (≤8)	monthly series per skill
GET	`/api/forecast/{skill}`	`dataset`	history, forecast, interval, metrics, caveat
GET	`/api/families`	—	10 families, members, recommendation, warnings
GET	`/api/related/{skill}`	—	substitutes and complements with scores
POST	`/api/stack/analyse`	JSON `{skills: []}`	verdicts with evidence
POST	`/api/upload`	multipart CSV, `mode`	upload id, summary, detected columns
GET	`/api/upload/limits`	—	max bytes, max rows, min months
GET	`/api/roles`	—	9 role profiles with confidence flags
GET	`/api/roles/matrix`	—	skill-by-role share matrix
GET	`/api/live/status`	—	last fetch, per-source counts, months held

Interactive documentation is generated by FastAPI at `/docs`.

Appendix B — Skill Dictionary

120 skills across 9 categories.

CATEGORY	COUNT	EXAMPLES
Programming Languages	18	Python, SQL, R, Java, Scala, C++, C#, Go, SAS
Databases & Warehouses	17	PostgreSQL, MySQL, SQL Server, Oracle, Snowflake, Redshift
Data Engineering & Big Data	16	ETL, Spark, Airflow, Kafka, dbt, Hadoop, Databricks
AI & Machine Learning	15	Machine Learning, Deep Learning, NLP, PyTorch, Scikit-learn
BI & Visualization	14	Tableau, Power BI, Looker, Qlik, Matplotlib, DAX
DevOps & Version Control	11	Git, GitHub, Docker, Kubernetes, CI/CD, Linux
Office & Collaboration	11	Microsoft Excel, PowerPoint, Jira, Confluence, SharePoint
Generative AI	10	LLM, Generative AI, prompt engineering, RAG
Cloud Platforms	8	AWS, Azure, GCP, Amazon S3, Azure Synapse

Special handling:

MECHANISM	ENTRY	EFFECT
`canonical_not_matched`	Go	Only `golang`, `go programming`, `go language` match
`blocked_next_tokens`	Microsoft Excel	Rejected before `at`, `in`, `as`, `beyond`
Longest-match	NoSQL	`no-sql` yields NoSQL only, never SQL

Appendix C — Substitute Families

FAMILY	MEMBERS	DATA-RANKED RECOMMENDATION
BI & dashboarding	8	Power BI
Cloud platform	3	AWS
Cloud data warehouse	5	Snowflake
Relational database	4	PostgreSQL
Deep learning framework	2	PyTorch
Statistics package	3	SAS
Python plotting	4	Matplotlib
NoSQL store	2+	MongoDB
Git hosting	2+	GitHub
ETL tooling	5	Fivetran

Ranking weights: historical demand 0.28, live demand 0.32, trend 0.22, reach 0.13, volatility penalty 0.05. Disagreement is flagged at a rank gap of 2 or more.

Deliberately excluded: Spark/Hadoop, as ambiguous. Explicitly protected as non-substitutes: Docker/Kubernetes, Python/Pandas, Python/NumPy, TensorFlow/Keras, SQL/PostgreSQL, Machine Learning/Deep Learning, ETL/Data Warehouse.

Appendix D — Monthly Data Quality

MONTH	POSTINGS	MEDIAN DESCRIPTION CHARS	ANALYSIS-GRADE
2022-11	2,123	3,122	yes
2022-12	3,272	3,123	yes
2023-01	3,646	3,542	yes
2023-02	2,779	3,212	yes
2023-03	2,692	3,165	yes
2023-04	2,455	3,135	yes
2023-05	2,317	2,841	yes
2023-06	2,333	2,830	yes
2023-07	2,533	3,130	yes
2023-08	2,988	3,569	yes
2023-09	3,042	2,752	yes
2023-10	3,321	2,676	yes
2023-11	2,595	3,040	yes
2023-12	2,338	3,169	yes
2024-01	2,292	2,971	yes
2024-02	2,521	2,851	yes
2024-03	2,100	2,908	yes
2024-04	1,689	2,599	yes
2024-05	1,731	2,504	yes
2024-06	1,613	2,541	yes
2024-07	1,417	2,434	yes
2024-09	272	4,744	**no** — low volume
2024-10	805	2,901	**no** — low volume
2024-11	492	3,789	**no** — low volume
2024-12	693	2,170	**no** — low volume
2025-01	1,302	1,713	**no** — truncated
2025-02	1,220	1,557	**no** — truncated
2025-03	1,179	1,601	**no** — truncated
2025-04	516	1,672	**no** — both

Thresholds: minimum 1,000 postings, minimum 2,000 median description characters. August 2024 is absent from the source data entirely.

The truncation pattern is visible in the final four rows: description length drops below 1,700 characters while volume remains adequate. This is the artifact that produced the false 29-declining result described in Chapter 9.

Appendix E — Glossary

Analysis-grade month — A month passing both quality thresholds, eligible to contribute to trends and forecasts.

Backtest — Holding out recent observations, fitting on the rest, and measuring prediction error against what actually happened.

Complement — A skill that works together with another; learning both adds value. Docker and Kubernetes.

Conditional probability — Here, P(skill | stack): the share of postings wanting the user's stack that also want a given skill.

Hit rate — The share of postings from which at least one skill was extracted.

MAE (mean absolute error) — Average size of prediction error, in percentage points.

MAPE (mean absolute percentage error) — Error as a percentage of the actual value; misleading when actual values are near zero.

Naive baseline — Predicting that the next value equals the last observed value. The standard reference for whether a forecast model has learned anything.

Parquet — Columnar binary file format with compression and preserved types.

PhraseMatcher — spaCy component matching token sequences, avoiding regex word-boundary failures on `C++`, `C#` and `CI/CD`.

PPMI (positive pointwise mutual information) — A co-occurrence measure of how much more often two items appear together than independence predicts, floored at zero.

Prophet — Open-source forecasting library producing trend, optional seasonality, and uncertainty intervals.

Reach — The number of distinct role families requesting a skill.

Substitute — A skill doing the same job as another; learning both largely duplicates effort. Tableau and Power BI.

Train offline / serve online — Architecture in which expensive computation runs ahead of time and the serving layer reads precomputed results.

Weak supervision — Using an existing imperfect labeller, such as a dictionary, to generate training data for a learned model.

End of report.